

ratvalue

Manual do Usuário

Framework de Valuation Damodaran em C++23

LAFI — Laboratório de Avaliação de Firmas e Investimentos
UFPel — Universidade Federal de Pelotas

Junho de 2026 v1.0

Sumário

Resumo	2
1 Introdução	3
1.1 Motivação	3
1.2 Escopo	3
1.3 Dependências e Build	3
2 Infraestrutura: ratmoney e tipos internos	5
2.1 ratmoney: aritmética monetária exata	5
2.2 types.h: erros e projeção	6
2.3 detail.h: aritmética racional interna	6
3 Normalização de Resultados	7
3.1 Teoria	7
3.2 API	7
3.3 Observações	8
4 Custo de Capital	9
4.1 CAPM com Risco-País	9
4.1.1 Modelo	9
4.1.2 API	9
4.2 Beta: Hamada e Bottom-Up	10
4.2.1 Equação de Hamada	10
4.2.2 Beta Bottom-Up	10
4.2.3 API	10
4.3 Kd Sintético	11
4.3.1 Metodologia	11
4.3.2 Tabela resumida (large cap)	11
4.3.3 API	12
4.4 WACC	12
4.4.1 Fórmula	12
4.4.2 API	12
5 Fluxo de Caixa Livre	13
5.1 FCFF — Free Cash Flow to Firm	13
5.1.1 Fórmula	13
5.1.2 API	13
5.2 FCFE — Free Cash Flow to Equity	13
5.2.1 Fórmula	13

5.2.2	Diferença conceitual: FCFF vs. FCFE	14
5.2.3	API	14
5.3	Crescimento Fundamental	15
5.3.1	Teoria	15
5.3.2	API	15
6	DCF — Desconto de Fluxos de Caixa	16
6.1	Modelo Padrão: FCFF e WACC	16
6.1.1	Estrutura multi-estágio	16
6.1.2	Implementação: double para desconto, Rational para TV	16
6.1.3	API	16
6.2	DDM — Dividend Discount Model	17
6.3	H-Model (Fuller & Hsia, 1984)	17
7	Valor Terminal	19
7.1	Consistência com ROIC	19
7.1.1	O problema do Gordon padrão	19
7.1.2	TV consistente (Damodaran cap. 12)	19
7.1.3	Auditoria de consistência	19
7.2	TV por Múltiplo	20
8	APV — Adjusted Present Value	21
8.1	Motivação	21
8.2	Modigliani-Miller (1963): dívida permanente	21
8.3	Miles-Ezzell (1980): dívida rebalanceada	21
9	EVA e Retornos em Excesso	23
9.1	Fundamento teórico	23
9.2	Interpretação econômica	23
9.3	API	23
10	Múltiplos Justificados	25
10.1	Fundamento	25
10.2	API	25
11	Estrutura Ótima de Capital	27
11.1	Algoritmo	27
11.2	API	27
12	Modelos Especializados	29
12.1	Valuation por Cenários: Startups e Firms em Transição	29
12.1.1	Motivação	29
12.1.2	API	29
12.2	Equity como Opção: Modelo de Merton	30
12.2.1	Fundamento	30
12.2.2	API	30
12.3	FCFE para Instituições Financeiras	31
12.3.1	Por que bancos são diferentes	31
12.3.2	Modelo	31

12.3.3 API	31
13 Exemplo Completo: Petrobras e Itaú Unibanco	32
13.1 Dados	32
13.2 Parâmetros de Custo de Capital	32
13.3 Crescimento e FCFF	33
13.4 Resumo dos Resultados	33
13.5 Banco Itaú Unibanco (ITUB4)	33
14 Fluxo de Dados entre Módulos	35
14.1 Visão geral	35
14.2 Consistência entre modelos	35
A Tabela de Ratings e Spreads (Damodaran 2023)	36
B Referências e Leitura Adicional	38

Resumo

ratvalue é uma biblioteca C++23 que implementa o framework completo de valuation de Aswath Damodaran, desde a normalização de resultados até modelos especializados para startups, firmas em dificuldade e instituições financeiras. O projeto é construído sobre **ratmoney**, que garante aritmética monetária com precisão racional exata (`int64_t` + GCD sobre `__int128`).

O manual cobre:

- Fundamentos teóricos de cada modelo de valuation;
- Especificação completa da API em C++23;
- Exemplos numéricos verificáveis;
- Exemplo real com Petrobras (PETR4) e Itaú Unibanco (ITUB4).

Pré-requisitos do leitor: familiaridade com C++17+, álgebra linear básica e conceitos de finanças corporativas ao nível de um curso de graduação.

Capítulo 1

Introdução

1.1 Motivação

Modelos de valuation são o núcleo analítico de toda decisão de alocação de capital. O framework de Damodaran, desenvolvido ao longo de três décadas em *Investment Valuation* (3ª ed., 2012) e *The Dark Side of Valuation* (3ª ed., 2018), é a referência mais rigorosa e completa disponível.

Implementações existentes concentram-se em Python/Excel, que têm dois problemas para análise quantitativa séria:

1. **Precisão numérica:** float64 não representa frações decimais exatamente. O erro acumula em cálculos encadeados.
2. **Performance:** Python possui GIL e overhead de boxing; Excel não é versionável nem auditável como código.

ratvalue resolve ambos: aritmética monetária exata via **ratmoney**, e performance de C++ compilado com `-O2`.

1.2 Escopo

A tabela [1.1](#) lista todos os módulos implementados.

1.3 Dependências e Build

Dependências:

- **ratmoney** ≥ 0.1 (instalado em `/usr/local`)
- **GCC** ≥ 13 ou **Clang** ≥ 17 (C++23)
- **CMake** ≥ 3.20
- **GTest** (para os testes)

Listing 1.1: Compilação e execução dos testes

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
ctest --test-dir build --output-on-failure
```

Tabela 1.1: Módulos do ratvalue

Arquivo	Categoria	Conteúdo
normalize	Dados	Normalização EBIT (histórica / margem)
capm	Custo de capital	CAPM com CRP (mercados emergentes)
beta	Custo de capital	Hamada; beta bottom-up
kd	Custo de capital	Kd sintético via ICR rating
wacc	Custo de capital	WACC
capital_structure	Estrutura	Estrutura ótima de capital
fcff	Fluxos	FCFF; projeção multi-estágio
fcfe	Fluxos	FCFE; DCF por equity
growth	Crescimento	ROIC, RR, crescimento fundamental
dcf	Valuation	DCF (FCFF WACC EV)
ddm	Valuation	DDM multi-estágio; H-Model
terminal	Valuation	TV consistente; TV por múltiplo
apv	Valuation	APV (MM e Miles-Ezzell)
excess_returns	Análise	EVA / Excess Returns
relative	Análise	P/L, P/VP, EV/EBITDA justificados
startup	Especializados	Valuation por cenários
distress	Especializados	Equity como opção (Merton)
bank	Especializados	FCFE para bancos (capital regulatório)

Capítulo 2

Infraestrutura: ratmoney e tipos internos

2.1 ratmoney: aritmética monetária exata

Toda unidade monetária em **ratvalue** é representada pelo tipo `ratmoney::Currency`, que armazena:

- **units**: contagem em menor denominação (*centavos*, para BRL e USD com `precision=2`);
- **rate**: valor de 1 unidade desta moeda em unidades de uma moeda-base implícita (tipo `Rational`);
- **description**: nome, símbolo e precisão decimal.

O tipo `ratmoney::Rational` representa um número racional p/q na forma canônica: $q > 0$, $\gcd(|p|, q) = 1$. Toda aritmética usa `__int128` como intermediário, eliminando overflow silencioso.

Listing 2.1: Criação de moedas no ratvalue

```
1 ratmoney::CurrencyDescription BRL{"BRL", "R$", 2};
2 ratmoney::Rational UNIT_RATE{1, 1};
3
4 // R$1 bilhão = 1e11 centavos
5 Currency B(int64_t bilhoes) {
6     return Currency{bilhoes * 100'000'000'000LL, UNIT_RATE, BRL};
7 }
8
9 Currency ebit = B(145);    // R$145 bilhoes
```

Atenção

A identidade de denominação é determinada por `CurrencyDescription`. Dois objetos `Currency` só podem ser somados ou comparados se tiverem a *mesma descrição*. O campo `rate` serve para *conversão entre moedas*, não para identificar a denominação. `ratvalue` valida consistência de moeda em todas as funções relevantes e retorna `ValuationError::CurrencyMismatch` em caso de violação.

2.2 types.h: erros e projeção

Listing 2.2: Tipos fundamentais do ratvalue

```

1 enum class ValuationError {
2     InvalidInput,      // entrada inválida (negativa, zero onde
                        // proibido...)
3     WACCLEGrowth,     // WACC <= g: denominador de Gordon seria <= 0
4     Overflow,         // overflow aritmético em Rational
5     MoneyError,       // CurrencyError do ratmoney
6     CurrencyMismatch, // inputs monetários em denominações diferentes
7 };
8
9 struct ProjectionStage {
10     ratmoney::Rational growth_rate;
11     int periods;      // anos neste estágio
12 };

```

Todas as funções do ratvalue retornam `std::expected<T, ValuationError>`, forçando tratamento explícito de erros em tempo de compilação.

2.3 detail.h: aritmética racional interna

O cabeçalho interno `detail.h` expõe helpers que implementam as operações $+$, $-$, $\times$, $\div$ sobre `Rational` com intermediários `__int128`:

```

1 namespace ratvalue::detail {
2     expected<Rational, ValuationError> r_add(Rational a, Rational b);
3     expected<Rational, ValuationError> r_sub(Rational a, Rational b);
4     expected<Rational, ValuationError> r_mul(Rational a, Rational b);
5     expected<Rational, ValuationError> r_div(Rational a, Rational b);
6     expected<Rational, ValuationError> one_plus(Rational r); // 1 + r
7     expected<Rational, ValuationError> make_rational(__int128 n,
8         __int128 d);
9
10    bool    same_currency(const Currency& a, const Currency& b);
11    double  to_double(const Currency& c);    // unidades maiores (ex:
12        reais)
13    expected<Currency, ValuationError>
14        make_currency_from_double(double v, Rational rate,
15            const CurrencyDescription& desc);
16 }

```

Atenção

`detail.h` é um cabeçalho *interno*, não instalado. Código cliente não deve incluí-lo diretamente.

Capítulo 3

Normalização de Resultados

3.1 Teoria

Firmas cíclicas (petróleo, mineração, siderurgia, agro) apresentam EBIT volátil que distorce qualquer DCF baseado no ano mais recente. Damodaran (cap. 8) recomenda normalizar antes de avaliar, usando uma das duas abordagens:

Média histórica Calcula a média aritmética do EBIT dos últimos N anos ($N \geq 2$), capturando o “EBIT do ciclo médio”:

$$EBIT_{\text{norm}} = \frac{1}{N} \sum_{i=1}^N EBIT_i$$

Margem normalizada Aplica uma margem operacional histórica (ou setorial) sobre a receita atual. Útil quando a receita é estável mas a margem oscila:

$$EBIT_{\text{norm}} = \text{Receita atual} \times \text{Margem normalizada}$$

3.2 API

```
1 enum class NormalizationMethod {
2     HistoricalAverage, // método 1
3     NormalizedMargin, // método 2
4 };
5
6 struct NormalizationInputs {
7     NormalizationMethod method;
8     std::vector<ratmoney::Currency> historical_ebit; // >= 2 para
9         Average
10    ratmoney::Currency current_revenue; // para
11        Margin
12    ratmoney::Rational normalized_margin; // para
13        Margin
14 };
15
16 expected<Currency, ValuationError>
17 normalize_ebit(const NormalizationInputs& inputs);
```

Exemplo: Petrobras — média histórica do ciclo

```
auto result = normalize_ebit(NormalizationInputs{
    .method = NormalizationMethod::HistoricalAverage,
    .historical_ebit = {B(80), B(95), B(145), B(165), B(145)},
    // EBIT 20192023 em R$ bilhões
    .current_revenue = B(500), // nao usado neste metodo
    .normalized_margin = {0, 1},
});
// result->units() / 1e11 = R$126B (média)
```

3.3 Observações

A aritmética usa `Currency::add` acumulado e `Currency::scale` com fator $\{1, N\}$. O arredondamento é *HalfEven* (padrão bancário), de modo que a soma exata é preservada e o erro de arredondamento é de no máximo 1 centavo.

Capítulo 4

Custo de Capital

4.1 CAPM com Risco-País

4.1.1 Modelo

Em mercados emergentes, Damodaran (*Investment Valuation*, cap. 7) estende o CAPM adicionando um prêmio de risco-país ponderado pela exposição λ da empresa:

$$K_e = R_f + \beta \cdot \text{ERP}_{\text{maduro}} + \lambda \cdot \text{CRP} \quad (4.1)$$

Onde:

- R_f : taxa livre de risco global (*T-Bond 10Y* americano, expurgado do componente de inflação diferencial se necessário);
- $\text{ERP}_{\text{maduro}}$: prêmio de risco do mercado de referência (geralmente EUA; Damodaran publica estimativas anuais);
- CRP: prêmio de risco-país, calculado como $\text{spread soberano} \times (\sigma_{\text{ações}} / \sigma_{\text{bônus}})$;
- λ : exposição da empresa ao risco-país. $\lambda = 1$ para empresas com receitas inteiramente domésticas; $\lambda < 1$ para exportadoras ou multinacionais.

Definição: Country Risk Premium (CRP)

$$\text{CRP} = \text{Spread soberano} \times \frac{\sigma_{\text{ações mercado emergente}}}{\sigma_{\text{bônus soberano}}}$$

Para o Brasil em 2024: spread EMBI+ $\approx 2\%$; fator de volatilidade $\approx 1,5$; CRP $\approx 3\%$.

4.1.2 API

```
1 struct CAPMInputs {
2     Rational risk_free_rate;
3     Rational beta;
4     Rational equity_risk_premium;           // ERP mercado maduro
5     Rational country_risk_premium{0, 1};    // CRP (padrão: 0)
6     Rational lambda{1, 1};                  // exposição país (
7     Rational lambda{1, 1};                  // padrão: 1)
8 }
```

```

8
9 expected<Rational, ValuationError>
10 cost_of_equity(const CAPMInputs& inputs);

```

Todo o cálculo é feito em aritmética racional exata. Para $R_f = 5\% = \frac{1}{20}$, $\beta = 1,30 = \frac{13}{10}$, $\text{ERP} = 7\% = \frac{7}{100}$, $\text{CRP} = 3\% = \frac{3}{100}$, $\lambda = 1$:

$$K_e = \frac{1}{20} + \frac{13}{10} \cdot \frac{7}{100} + 1 \cdot \frac{3}{100} = \frac{50}{1000} + \frac{91}{1000} + \frac{30}{1000} = \frac{171}{1000} = 17,1\%$$

4.2 Beta: Hamada e Bottom-Up

4.2.1 Equação de Hamada

A alavancagem financeira amplifica o risco sistemático:

$$\beta_L = \beta_U \cdot \left[1 + (1 - t) \cdot \frac{D}{E} \right] \quad (4.2)$$

Inversamente:

$$\beta_U = \frac{\beta_L}{1 + (1 - t) \cdot D/E} \quad (4.3)$$

4.2.2 Beta Bottom-Up

O beta histórico (regressão de retornos) é ruidoso e reflete a estrutura de capital passada. Damodaran recomenda o *beta bottom-up*:

1. Coletar β_L de n empresas comparáveis (mesmo setor);
2. Desalavancar cada comparável via eq. (4.3);
3. Calcular a média: $\bar{\beta}_U = n^{-1} \sum_i \beta_{U,i}$;
4. Re-alavancar para a estrutura-alvo via eq. (4.2).

4.2.3 API

```

1 // Alavancar
2 expected<Rational, ValuationError>
3 lever_beta(Rational unlevered_beta, Rational debt_to_equity, Rational
4   tax_rate);
5
6 // Desalavancar
7 expected<Rational, ValuationError>
8 unlever_beta(Rational levered_beta, Rational debt_to_equity, Rational
9   tax_rate);
10
11 struct BottomUpBetaInputs {
12     vector<Rational> comparable_levered_betas;
13     vector<Rational> comparable_debt_to_equity;
14     Rational         comparable_tax_rate;
15     Rational         target_debt_to_equity;
16     Rational         target_tax_rate;
17 };

```

```

16
17 expected<Rational, ValuationError>
18 bottom_up_beta(const BottomUpBetaInputs& inputs);

```

Atenção

A média dos β_U em `bottom_up_beta` é calculada em `double`, pois betas de mercado são estimativas sem precisão exata. A re-alavancagem final retorna `Rational` com 4 casas decimais.

4.3 Kd Sintético

4.3.1 Metodologia

Empresas sem bônus negociados não têm K_d observável. Damodaran propõe estimá-lo a partir do *índice de cobertura de juros* (ICR):

$$\text{ICR} = \frac{\text{EBIT}}{\text{Despesa Financeira}} \longrightarrow \text{rating sintético} \longrightarrow K_d = R_f + \text{spread}(\text{rating})$$

A tabela de ICR $\rightarrow$ rating é publicada anualmente em <https://pages.stern.nyu.edu/~adamodar/>. O `ratvalue` implementa as tabelas de 2023 para *large cap* e *small cap* (thresholds mais exigentes para empresas menores).

4.3.2 Tabela resumida (large cap)

Tabela 4.1: ICR Rating Spread (large cap, Damodaran 2023)

ICR mínimo	Rating	Spread (bps)
> 8,50	Aaa/AAA	63
6,50	Aa2/AA	78
5,50	A1/A+	98
4,25	A2/A	108
3,00	A3/A-	122
2,50	Baa2/BBB	156
2,25	Ba1/BB+	200
2,00	Ba2/BB	240
1,75	B1/B+	350
1,50	B2/B	375
1,25	B3/B-	500
0,80	Caa/CCC	600
0,65	Ca2/CC	700
0,20	C2/C	800
< 0,20	D2/D	1200

4.3.3 API

```

1  enum class CreditRating : uint8_t {
2      AAA, AA, A_plus, A, A_minus, BBB, BB_plus, BB,
3      B_plus, B, B_minus, CCC, CC, C, D
4  };
5
6  struct SyntheticKdInputs {
7      Currency ebit;
8      Currency interest_expense;
9      Rational risk_free_rate;
10     bool      large_firm{true};
11 };
12
13 struct SyntheticKdResult {
14     Rational      cost_of_debt;          // Kd = Rf + spread (Rational
15         exato)
16     Rational      default_spread;
17     CreditRating  rating;
18     double        interest_coverage;
19 };
20
21 expected<SyntheticKdResult, ValuationError>
22 synthetic_cost_of_debt(const SyntheticKdInputs& inputs);
23
24 // versão direta (para o otimizador interno)
25 expected<SyntheticKdResult, ValuationError>
26 synthetic_cost_of_debt_from_icr(double icr, Rational rf, bool
27     large_firm);

```

4.4 WACC

4.4.1 Fórmula

$$\text{WACC} = \frac{E}{V}K_e + \frac{D}{V}K_d(1 - t) \quad (4.4)$$

Onde $V = E + D$.

4.4.2 API

```

1  struct WACCInputs {
2      Rational equity_weight;    // E/V
3      Rational debt_weight;     // D/V
4      Rational cost_of_equity;   // Ke
5      Rational cost_of_debt;     // Kd
6      Rational tax_rate;        // t
7  };
8
9  expected<Rational, ValuationError>
10 compute_wacc(const WACCInputs& inputs);

```

Toda a aritmética é racional exata. Para os parâmetros do Exemplo Petrobras: $E_w = 2/3$, $K_e = 4629/40000$, $D_w = 1/3$, $K_d = 578/10000$, $t = 17/50$:

$$\text{WACC} = \frac{2}{3} \cdot \frac{4629}{40000} + \frac{1}{3} \cdot \frac{578}{10000} \cdot \frac{33}{50} = \frac{44933}{500000} \approx 8,99\%$$

Capítulo 5

Fluxo de Caixa Livre

5.1 FCFF — Free Cash Flow to Firm

5.1.1 Fórmula

$$\text{FCFF} = \text{EBIT}(1 - t) + D\&A - \text{CapEx} - \Delta\text{CGN} \quad (5.1)$$

Onde ΔCGN é a variação no capital de giro não caixa (Damodaran usa $\Delta\text{NWC} = \Delta\text{CG}$).

5.1.2 API

```
1 struct FCFFInputs {
2     Currency ebit;
3     Rational tax_rate;
4     Currency depreciation_amortization;
5     Currency capex;
6     Currency delta_nwc;
7 };
8
9 expected<Currency, ValuationError>
10 compute_fcff(const FCFFInputs& inputs);
11
12 struct FCFFProjection {
13     Currency base_fcff;
14     std::vector<ProjectionStage> stages;
15 };
16
17 // Retorna [FCFF_1, FCFF_2, ..., FCFF_N] (base não incluído)
18 expected<std::vector<Currency>, ValuationError>
19 project_fcff(const FCFFProjection& proj);
```

A projeção é exata: cada período aplica `Currency::scale` com fator $(1 + g)$ representado como `Rational`.

5.2 FCFE — Free Cash Flow to Equity

5.2.1 Fórmula

$$\text{FCFE} = \text{NI} - (1 - \delta)(\text{CapEx} - D\&A + \Delta\text{CGN}) + \Delta D \quad (5.2)$$

Onde $\delta = D/(D + E)$ é o *debt ratio*. A interpretação: o acionista financia apenas a parcela $(1 - \delta)$ do reinvestimento líquido; o restante é coberto por nova dívida.

5.2.2 Diferença conceptual: FCFF vs. FCFE

Tabela 5.1: Comparação FCFF e FCFE

Dimensão	FCFF	FCFE
Base	EBIT (pré-juros)	Lucro Líquido (pós-juros)
Desconto	WACC	K_e
Resultado	Enterprise Value	Equity Value (direto)
Alavancagem	Capturada no WACC	Capturada no FCFE

Atenção

FCFF e FCFE produzem o mesmo valor de equity quando os inputs são consistentes (mesma estrutura de capital implícita). Discrepâncias indicam inconsistência nas hipóteses, não erros no código.

5.2.3 API

```

1 struct FCFEInputs {
2     Currency net_income;
3     Rational debt_ratio;           // delta = D/(D+E)
4     Currency capex;
5     Currency depreciation_amortization;
6     Currency delta_nwc;
7     Currency net_new_debt;        // pode ser negativo
8 };
9
10 expected<Currency, ValuationError>
11 compute_fcfe(const FCFEInputs& inputs);
12
13 struct FCFEDCFInputs {
14     std::vector<Currency> projected_fcfe;
15     Rational cost_of_equity;
16     Rational terminal_growth;
17     int64_t shares_outstanding;
18 };
19
20 struct FCFEDCFResult {
21     Currency equity_value;
22     Currency equity_value_per_share;
23     Currency pv_fcfe;
24     Currency terminal_value;
25     Currency pv_terminal_value;
26 };
27
28 expected<FCFEDCFResult, ValuationError>
29 compute_fcfe_dcf(const FCFEDCFInputs& inputs);

```

5.3 Crescimento Fundamental

5.3.1 Teoria

Damodaran insiste que hipóteses de crescimento devem ser *ancoradas* em fundamentos. Crescimento “gratuito” (sem reinvestimento) é um erro comum que infla artificialmente o valor.

$$g_{\text{firma}} = \text{ROIC} \times \text{RR} \quad (5.3)$$

$$g_{\text{equity}} = \text{ROE} \times (1 - \text{payout}) \quad (5.4)$$

Onde:

$$\text{ROIC} = \frac{\text{NOPAT}}{\text{IC}} \quad \text{RR} = \frac{\text{CapEx} - D\&A + \Delta\text{CGN}}{\text{NOPAT}}$$

5.3.2 API

```

1 // ROIC = NOPAT / Invested Capital
2 struct ROICInputs { Currency nopat; Currency invested_capital; };
3 expected<Rational, ValuationError> compute_roic(const ROICInputs&);
4
5 // RR = (CapEx - DA + dNWC) / NOPAT
6 struct ReinvestmentRateInputs {
7     Currency capex, depreciation_amortization, delta_nwc, nopat;
8 };
9 expected<Rational, ValuationError>
10 compute_reinvestment_rate(const ReinvestmentRateInputs&);
11
12 // g = ROIC x RR
13 expected<Rational, ValuationError>
14 fundamental_growth_firm(Rational roic, Rational reinvestment_rate);
15
16 // g = ROE x retention
17 expected<Rational, ValuationError>
18 fundamental_growth_equity(Rational roe, Rational retention_ratio);

```

Capítulo 6

DCF — Desconto de Fluxos de Caixa

6.1 Modelo Padrão: FCFF e WACC

6.1.1 Estrutura multi-estágio

O DCF padrão de Damodaran divide o horizonte em dois estágios:

1. **Período explícito** ($t = 1, \dots, N$): FCFFs projetados descontados individualmente.
2. **Perpetuidade estável** ($t > N$): valor terminal (TV) calculado via Gordon Growth.

$$EV = \sum_{t=1}^N \frac{FCFF_t}{(1 + WACC)^t} + \frac{TV_N}{(1 + WACC)^N} \quad (6.1)$$

$$TV_N = \frac{FCFF_N \cdot (1 + g)}{(WACC - g)} \quad (6.2)$$

$$\text{Equity} = EV - \text{Dívida Líquida} \quad (6.3)$$

$$P_{\text{ação}} = \frac{\text{Equity}}{n_{\text{ações}}} \quad (6.4)$$

6.1.2 Implementação: double para desconto, Rational para TV

Atenção

O fator de desconto $(1 + r)^{-t}$ é irracional para $t > 1$ na maioria dos casos. O `ratvalue` usa `double` *apenas* para as potências; todas as demais operações são racionais exatas. Em particular, o multiplicador do TV é um **Rational** exato:

$$TV_{\text{mul}} = \frac{(1 + g)/g_{\text{den}}}{(WACC - g)} = \frac{(g_{\text{den}} + g_{\text{num}}) \cdot WACC_{\text{den}}}{WACC_{\text{num}} \cdot g_{\text{den}} - g_{\text{num}} \cdot WACC_{\text{den}}}$$

6.1.3 API

```

1 struct DCFInputs {
2     std::vector<Currency> projected_fcff;
3     Rational wacc;
4     Rational terminal_growth;
5     Currency net_debt;
6     int64_t shares_outstanding;
7 };
8
9 struct DCFResult {
10     Currency enterprise_value;
11     Currency equity_value;
12     Currency equity_value_per_share;
13     Currency pv_fcff;
14     Currency terminal_value; // TV sem descontar (Rational exato)
15     Currency pv_terminal_value;
16 };
17
18 expected<DCFResult, ValuationError>
19 compute_dcf(const DCFInputs& inputs);

```

6.2 DDM — Dividend Discount Model

O DDM substitui FCFF por dividendos e desconta a K_e :

```

1 struct DDMInputs {
2     Currency base_dividend_per_share;
3     std::vector<ProjectionStage> stages;
4     Rational cost_of_equity;
5     Rational stable_growth;
6 };
7
8 expected<DDMResult, ValuationError>
9 compute_ddm(const DDMInputs& inputs);

```

6.3 H-Model (Fuller & Hsia, 1984)

O H-Model é uma fórmula fechada para crescimento que *declina linearmente* de g_H para g_L ao longo de $2H$ anos:

$$P = \frac{D_0 \cdot [(1 + g_L) + H \cdot (g_H - g_L)]}{K_e - g_L} \quad (6.5)$$

Todo o cálculo é *exato* em aritmética racional — não há desconto temporal envolvido.

```

1 struct HModelInputs {
2     Currency base_dividend;
3     Rational high_growth; // g_H
4     Rational stable_growth; // g_L
5     Rational half_life; // H em anos (ex: {5,1})
6     Rational cost_of_equity;
7 };
8
9 struct HModelResult {

```

```
10     Currency intrinsic_value;  
11     Rational tv_multiplier;    // para auditoria  
12 };  
13  
14 expected<HModelResult, ValuationError>  
15 compute_h_model(const HModelInputs& inputs);
```

Capítulo 7

Valor Terminal

7.1 Consistência com ROIC

7.1.1 O problema do Gordon padrão

O TV pelo Gordon Growth implica uma taxa de reinvestimento:

$$RR_{\text{impl}} = \frac{g}{ROIC_{\text{estável}}}$$

Se o analista assume $g = 3\%$ mas $ROIC_{\text{estável}} = 8\%$, então $RR_{\text{impl}} = 37,5\%$. Se ele projetou os FCFFs com $RR = 30\%$, há uma descontinuidade no terminal que infla o TV.

7.1.2 TV consistente (Damodaran cap. 12)

$$TV = \frac{NOPAT_{\text{estável}} \cdot (1 - g/ROIC_{\text{estável}})}{WACC - g} \quad (7.1)$$

Usa o NOPAT do período terminal como base, derivando o FCFF estável pela RR consistente com o ROIC assumido.

7.1.3 Auditoria de consistência

```
1 struct TerminalConsistency {
2     Rational implied_reinvestment_rate; // g / ROIC_estavel
3     Rational return_spread;           // ROIC_estavel - WACC
4     bool value_creating;              // ROIC > WACC
5     bool reinvestment_feasible;       // 0 <= g/ROIC <= 1
6 };
7
8 TerminalConsistency check_terminal_consistency(
9     Rational wacc, Rational terminal_growth, Rational terminal_roic);
10
11 struct ConsistentTVInputs {
12     Currency stable_nopat;
13     Rational wacc;
14     Rational terminal_growth;
15     Rational terminal_roic;
16 };
17
18 expected<Currency, ValuationError>
19 consistent_terminal_value(const ConsistentTVInputs& inputs);
```


Capítulo 8

APV — Adjusted Present Value

8.1 Motivação

O APV separa explicitamente o valor operacional do benefício fiscal:

$$APV = V_U + PV(\text{Tax Shields}) \quad (8.1)$$

Onde V_U é o valor da firma *sem alavancagem*, obtido descontando os FCFFs ao custo do equity desalavancado K_u :

$$K_u = R_f + \beta_U \cdot \text{ERP} + \lambda \cdot \text{CRP}$$

O APV é preferível ao WACC quando a estrutura de capital varia ao longo do tempo (LBOs, recapitalizações, projetos com cronograma de amortização).

8.2 Modigliani-Miller (1963): dívida permanente

Para dívida permanente e tax shields com risco zero (desconto a R_f):

$$PV(\text{TS})_{\text{MM}} = t \cdot D \quad (8.2)$$

```
1 struct APVInputs {
2     std::vector<Currency> projected_fcff;
3     Rational             terminal_growth;
4     Rational             unlevered_cost_of_equity; // Ku
5     Currency             debt_market_value;       // D
6     Rational             tax_rate;                // t
7     Currency             net_debt;
8     int64_t              shares_outstanding;
9 };
10
11 expected<APVResult, ValuationError>
12 compute_apv(const APVInputs& inputs);
```

8.3 Miles-Ezzell (1980): dívida rebalanceada

Quando a firma mantém um D/V constante (rebalanceia a cada período), os tax shields têm o risco dos ativos e devem ser descontados a K_u , exceto o primeiro período (conhecido com certeza):

$$PV(TS)_{ME} = \frac{t \cdot K_d \cdot D \cdot (1 + K_u)}{(1 + K_d)(K_u - g)} \quad (8.3)$$

O multiplicador de Miles-Ezzell é calculado em aritmética racional exata via `detail::r_mul` e `detail::r_div`.

```

1 struct APVMilesEzzellInputs {
2     // ... todos os campos de APVInputs, mais:
3     Rational cost_of_debt;    // Kd (necessario para formula ME)
4 };
5
6 expected<APVResult, ValuationError>
7 compute_apv_miles_ezzell(const APVMilesEzzellInputs& inputs);

```

Exemplo: Comparação MM vs. ME (Petrobras)

$D = \text{R\$}250B$, $t = 34\%$, $K_d = 5,78\%$, $K_u \approx 11,57\%$, $g = 1,5\%$:

$$PV(TS)_{MM} = 0,34 \times 250 = \text{R\$}85B$$

$$PV(TS)_{ME} = \frac{0,34 \times 0,0578 \times 250 \times 1,1157}{1,0578 \times 0,1007} \approx \text{R\$}51B$$

O MM superestima o benefício fiscal em $\approx 40\%$ neste caso.

Capítulo 9

EVA e Retornos em Excesso

9.1 Fundamento teórico

O modelo de *excess returns* (Damodaran, cap. 13) decompõe o valor da firma em:

$$V = IC + PV(\text{EVA}) \quad (9.1)$$

Onde $\text{EVA}_t = (\text{ROIC}_t - \text{WACC}) \times \text{IC}_t$ e IC_t cresce com o reinvestimento. Para o estágio estável (single-stage, Gordon):

$$V = \text{IC} \cdot \frac{\text{ROIC} - g}{\text{WACC} - g} \quad (9.2)$$

Equivalência com DCF: quando os inputs são consistentes ($g = \text{ROIC} \times \text{RR}$ e $\text{FCFF} = \text{NOPAT}(1 - \text{RR})$), as equações (6.1) e (9.1) produzem o mesmo V . A decomposição do EVA revela o *valor dos ativos existentes* vs. o *valor do crescimento futuro*.

9.2 Interpretação econômica

- $\text{ROIC} > \text{WACC}$: a firma cria valor acima do custo de capital; $PV(\text{EVA}) > 0$.
- $\text{ROIC} = \text{WACC}$: crescimento *neutro*; $V = \text{IC}$.
- $\text{ROIC} < \text{WACC}$: cada real investido destrói valor; $PV(\text{EVA}) < 0$.

9.3 API

```
1 struct SingleStageERInputs {
2     Currency invested_capital;
3     Rational roic;
4     Rational wacc;
5     Rational terminal_growth;
6 };
7
8 struct ExcessReturnsResult {
9     Currency firm_value;
10    Currency asset_value;           // = IC
11    Currency pv_excess_returns;    // PV(EVA)
12 };
```

```
13
14 expected<ExcessReturnsResult, ValuationError>
15 excess_returns_value(const SingleStageERInputs& inputs);
16
17 struct MultiStageERInputs {
18     Currency          base_invested_capital;
19     Rational          roic;
20     std::vector<ProjectionStage> stages;
21     Rational          wacc;
22     Rational          terminal_roic;
23     Rational          terminal_growth;
24 };
25
26 expected<ExcessReturnsResult, ValuationError>
27 excess_returns_multistage(const MultiStageERInputs& inputs);
```

Capítulo 10

Múltiplos Justificados

10.1 Fundamento

Múltiplos de mercado (P/L, P/VP, EV/EBITDA) são frequentemente interpretados sem referência aos fundamentos. Damodaran (cap. 17–20) deriva o *múltiplo justificado*: o valor que seria correto dado o crescimento, a rentabilidade e o custo de capital da firma.

$$P/L = \frac{\text{payout}}{K_e - g_{\text{equity}}} \quad (10.1)$$

$$P/VP = \text{ROE} \times \frac{\text{payout}}{K_e - g_{\text{equity}}} \quad (10.2)$$

$$EV/EBITDA = \frac{(1 - t)(1 - \text{RR})}{\text{WACC} - g} \quad (10.3)$$

Onde $g_{\text{equity}} = \text{ROE} \times (1 - \text{payout})$ e $\text{RR} = g/\text{ROIC}$ (taxa de reinvestimento da firma).

10.2 API

```
1 struct JustifiedMultiplesInputs {
2     Rational cost_of_equity;
3     Rational wacc;
4     Rational roe;
5     Rational payout_ratio;
6     Rational roic;
7     Rational tax_rate;
8 };
9
10 struct JustifiedMultiples {
11     Rational pe_ratio;
12     Rational pb_ratio;
13     Rational ev_ebitda;
14     Rational g_equity;
15     Rational implied_reinvestment;
16 };
17
18 expected<JustifiedMultiples, ValuationError>
19 compute_justified_multiples(const JustifiedMultiplesInputs& inputs);
```

Exemplo: Petrobras — múltiplos justificados (ROE normalizado = 15%)

Com $K_e = 11,57\%$, $WACC = 8,99\%$, $ROE_{\text{norm}} = 15\%$, $\text{payout} = 60\%$, $ROIC = 16,63\%$, $t = 34\%$:

$$g = 15\% \times 40\% = 6\%$$

$$P/L = \frac{0,60}{0,1157 - 0,06} = 10,8\times \quad P/VP = 0,15 \times 10,8 = 1,6\times \quad EV/EBITDA = 14,1\times$$

Capítulo 11

Estrutura Ótima de Capital

11.1 Algoritmo

O custo de capital mínimo maximiza o valor da firma. O `ratvalue` varre razões de endividamento $d \in \{0/N, 1/N, \dots, (N-1)/N\}$ e para cada ponto:

1. Calcula $D/E = d/(1-d)$ como `Rational`;
2. Alavancar $\beta_U \rightarrow \beta_L$ via Hamada (eq. 4.2);
3. Calcula K_e via CAPM (eq. 4.1);
4. Itera K_d a partir do ICR:
 - (a) Interesse = $D \times K_d$
 - (b) ICR = EBIT/Interesse
 - (c) $K_d \leftarrow R_f + \text{spread}(\text{ICR})$
 - (d) Repetir até convergência ($< 10^{-9}$, usualmente 2–5 iterações)
5. Calcula WACC exato como `Rational`.

O ótimo é o ponto de mínimo WACC.

11.2 API

```
1 struct OptimalCapitalStructureInputs {
2     Rational    unlevered_beta;
3     Rational    risk_free_rate;
4     Rational    equity_risk_premium;
5     Rational    country_risk_premium{0, 1};
6     Rational    lambda{1, 1};
7     Rational    tax_rate;
8     Currency    ebit;           // para calcular ICR
9     Currency    firm_value;     // E+D (base para D = d * FV)
10    int          steps{10};
11    bool         large_firm{true};
12 };
13
14 struct CapitalStructurePoint {
```

```

15     Rational    debt_ratio;
16     Rational    levered_beta;
17     Rational    cost_of_equity;
18     Rational    cost_of_debt;
19     CreditRating rating;
20     double      interest_coverage;
21     Rational    wacc;
22 };
23
24 struct OptimalCapitalStructureResult {
25     std::vector<CapitalStructurePoint> schedule;
26     CapitalStructurePoint              optimal;
27 };
28
29 expected<OptimalCapitalStructureResult, ValuationError>
30 optimal_capital_structure(const OptimalCapitalStructureInputs& inputs
    );

```

Atenção

O modelo de Damodaran via ICR captura apenas o *custo de default via spread*. Custos indiretos de dificuldade financeira (perda de clientes, contratação subótima, distraction gerencial) não são modelados. Em firmas com EBIT alto relativo ao valor, o algoritmo pode indicar razões de endividamento elevadas que na prática seriam inviáveis.

Capítulo 12

Modelos Especializados

12.1 Valuation por Cenários: Startups e Firmas em Transição

12.1.1 Motivação

Empresas jovens ou em transição tecnológica têm distribuição de resultados fortemente assimétrica. Um único DCF “base” ignora essa dispersão. Damodaran (*Dark Side*, cap. 10–12) recomenda:

1. Definir n cenários (geralmente 3–5) com probabilidades;
2. Calcular o TV de cada cenário: $TV_i = FCFF_i \cdot mul_i$;
3. Descontar cada TV ao presente: $PV_i = TV_i / (1 + WACC_i)^T$;
4. Aplicar probabilidade de sobrevivência:

$$EV_{\text{ajustado}} = \left[\sum_i P_i \cdot PV_i \right] \cdot P(\text{sobrevive}) + V_{\text{falência}} \cdot [1 - P(\text{sobrevive})] \quad (12.1)$$

12.1.2 API

```
1 struct ValuationScenario {
2     std::string name;
3     Rational    probability;
4     Currency    terminal_fcff;
5     Rational    terminal_growth;
6     Rational    wacc;
7     int         years_to_terminal;
8 };
9
10 struct StartupValuationInputs {
11     std::vector<ValuationScenario> scenarios;
12     Rational survival_probability;
13     Currency failure_value;
14     Currency net_debt;
15     int64_t shares_outstanding;
```



```

16 };
17
18 expected<StartupValuationResult, ValuationError>
19 startup_valuation(const StartupValuationInputs& inputs);

```

A soma das probabilidades deve ser 1,0 (tolerância 10^{-6}); caso contrário, `InvalidInput` é retornado.

12.2 Equity como Opção: Modelo de Merton

12.2.1 Fundamento

Em firmas com dívida elevada, o equity comporta-se como uma *opção de compra* sobre o valor dos ativos, com strike igual ao valor de face da dívida (Merton, 1974):

$$d_1 = \frac{\ln(V/D) + (r + \sigma_V^2/2)T}{\sigma_V \sqrt{T}} \quad (12.2)$$

$$d_2 = d_1 - \sigma_V \sqrt{T} \quad (12.3)$$

$$E = V \cdot \mathcal{N}(d_1) - D \cdot e^{-rT} \cdot \mathcal{N}(d_2) \quad (12.4)$$

A probabilidade *risk-neutral* de default é $\mathcal{N}(-d_2)$. A distância ao default d_2 é o indicador de saúde financeira.

Atenção

V e σ_V são os ativos da firma, *não observáveis* diretamente. A estimação iterativa (resolver $\{E, \sigma_E\}$ com o sistema não-linear $E = \text{BS}(V, \sigma_V)$ e $\sigma_E E = \mathcal{N}(d_1) \sigma_V V$) não é implementada; o usuário fornece V e σ_V .

12.2.2 API

```

1 struct EquityAsOptionInputs {
2     double firm_value;           // V
3     double debt_face_value;     // D
4     double asset_volatility;    // sigma_V
5     double risk_free_rate;     // r (contínuo)
6     double debt_maturity;      // T em anos
7 };
8
9 struct EquityAsOptionResult {
10     double equity_value;
11     double debt_market_value;
12     double probability_default; // N(-d2)
13     double distance_to_default; // d2
14     double d1;
15 };
16
17 EquityAsOptionResult                               // sem expected: sempre
    definido
18 equity_as_option(const EquityAsOptionInputs& inputs) noexcept;

```

A CDF normal é implementada via: $\mathcal{N}(x) = \frac{1}{2} \operatorname{erfc}(-x/\sqrt{2})$ (função `std::erfc` de `<cmath>`).

12.3 FCFE para Instituições Financeiras

12.3.1 Por que bancos são diferentes

- A dívida (depósitos, CDBs) é *matéria-prima*, não fonte de financiamento capex — não faz sentido calcular FCFF ou WACC.
- O reinvestimento é regulado: Basileia III exige capital mínimo sobre os ativos ponderados pelo risco (RWA).
- O custo de capital relevante é apenas K_e .

12.3.2 Modelo

$$\text{FCFE}_{\text{banco}} = NI \times (1 - \text{RR}_{\text{equity}}) \quad (12.5)$$

$$\text{RR}_{\text{equity}} = \frac{\Delta \text{RWA} \times \text{target Tier1}}{NI}$$

12.3.3 API

```

1 struct BankFCFEInputs {
2     Currency net_income;
3     Rational equity_reinvestment_rate;
4 };
5
6 expected<Currency, ValuationError>
7 compute_bank_fcfe(const BankFCFEInputs& inputs);
8
9 struct RegulatoryCapitalInputs {
10     Currency net_income;
11     Currency current_rwa;
12     Currency projected_rwa;
13     Rational target_tier1_ratio;
14 };
15
16 expected<Rational, ValuationError>
17 bank_equity_reinvestment_rate(const RegulatoryCapitalInputs& inputs);

```

Para o DCF de um banco, usa-se `compute_fcfe_dcf` com os FCFEs gerados por `compute_bank_fcfe`.

Capítulo 13

Exemplo Completo: Petrobras e Itaú Unibanco

13.1 Dados

Tabela 13.1: Petrobras S.A. — dados aproximados 2023

Item	R\$ bilhões	Código
Receita líquida	500	B(500)
EBIT histórico 2019–2023	80, 95, 145, 165, 145	B(80) . . .
EBIT normalizado (média)	126	—
D&A	45	B(45)
CapEx	70	B(70)
Δ CGN	5	B(5)
Lucro Líquido	124	B(124)
Despesa Financeira	22	B(22)
Dívida Líquida	250	B(250)
Capital Investido	500	B(500)
Ações	13 bilhões	13e9

13.2 Parâmetros de Custo de Capital

- $R_f = 5\%$ (T-Bond 10Y *stripped*);
- ERP = 5% (mercado americano, Damodaran 2024);
- CRP = 3% (Brasil, EMBI+ ajustado por volatilidade);
- $\lambda = 0,5$ (Petrobras: $\approx 50\%$ receita no exterior);
- $\beta_U = 0,8$ (setor O&G, bottom-up a partir de Shell, Exxon, BP, Chevron, Equinor com $t_{\text{setor}} = 25\%$);
- $D/E = 250/500 = 0,5$; $t = 34\%$.

Resultados de custo de capital: $\beta_L \approx 1,014$; $K_e \approx 11,57\%$; ICR= $145/22 = 6,6\times \rightarrow$ AA rating $\rightarrow K_d = 5,78\%$; WACC $\approx 8,99\%$ (racional exato: $44933/500000$).

13.3 Crescimento e FCFF

$$\text{FCFF}_0 = 126 \times 0,66 + 45 - 70 - 5 = \text{R\$}53,2B$$

$$\text{ROIC} = 83,2/500 = 16,63\%$$

$$\text{RR} = 30/83,2 = 36,08\%$$

$$g_{\text{fund}} = 16,63\% \times 36,08\% = 6\%$$

Para o DCF usa-se $g_{\text{explícito}} = 3\%$ por 5 anos e $g_{\text{terminal}} = 1,5\%$.

13.4 Resumo dos Resultados

Tabela 13.2: Comparativo de modelos — Petrobras (PETR4)

Modelo	Preço justo	Módulo
DCF (FCFF, Gordon TV)	R\$39,88	dcf
FCFE (equity direto)	R\$85,72	fcfe
APV Modigliani-Miller	R\$31,13	apv
APV Miles-Ezzell	R\$28,55	apv
H-Model (dividendos)	R\$54,11	ddm
Cenários (transição E)	R\$24,42	startup
EVA / Excess Returns	R\$58,51	excess_returns
Cotação mercado (2024)	R\$38–42	—

Atenção

A discrepância entre modelos é *esperada e informativa*:

- O **FCFE** usa NI de 2023 (R\$124B) sem normalização — inflado pelo ciclo de petróleo elevado;
- O **APV** não inclui benefício da dívida nas taxas de crescimento, de modo que o TV é menor;
- O modelo de **cenários** penaliza fortemente a transição energética acelerada (20% de probabilidade de FCFF cair para R\$40B);
- O **DCF/FCFF normalizado** (R\$39,88) é o mais consistente com a cotação de mercado por usar EBIT médio do ciclo.

13.5 Banco Itaú Unibanco (ITUB4)

A alta $\text{RR}_{\text{equity}}$ reflete o crescimento rápido do crédito brasileiro: o banco precisa reter $\approx 56\%$ dos lucros para suportar a expansão regulatória dos ativos.

Tabela 13.3: Itaú Unibanco — dados aproximados 2023

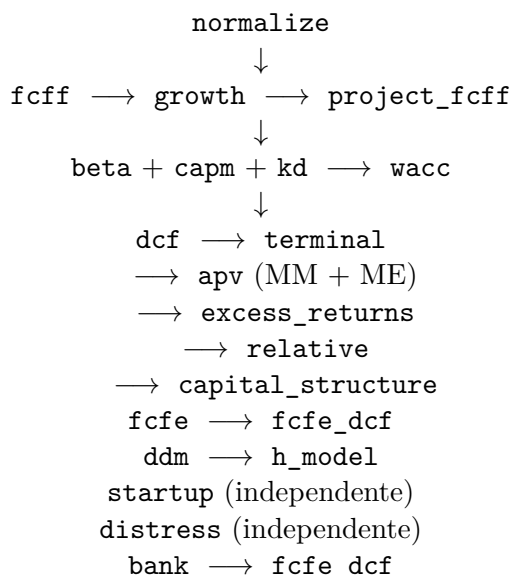
Item	Valor
Lucro Líquido	R\$35B
RWA atual	R\$1.800B
RWA projetado	R\$1.944B (+8%)
Target Tier1	13,5%
Δ Capital regulatório	$R\$144B \times 13,5\% = R\$19,4B$
RR_{equity}	$R\$19,4B / R\$35B = 55,5\%$
$FCFE_{\text{banco}}$	$R\$35B \times 44,5\% = R\$15,6B$
Preço justo (DCF)	R\$22,61/ação

Capítulo 14

Fluxo de Dados entre Módulos

14.1 Visão geral

A figura abaixo ilustra o fluxo típico de uma análise completa:



14.2 Consistência entre modelos

Para que FCFF-DCF e FCFE-DCF produzam o mesmo preço por ação:

1. $FCFE = FCFF - \text{Juros}(1 - t) + \Delta D$;
2. A taxa de crescimento usada em ambos deve ser a mesma;
3. O ΔD deve ser consistente com a razão D/V ao longo do tempo.

Para que APV e WACC-DCF produzam o mesmo EV:

- WACC presume alavancagem constante (ajusta WACC a cada período);
- APV presume cronograma de dívida predeterminado.
- Em geral, APV é mais flexível; WACC é mais simples para estrutura estável.

Apêndice A

Tabela de Ratings e Spreads (Damodaran 2023)

Esquerda: *large cap.* Direita: *small cap.*

Tabela A.1: Large Cap

Rating	ICR mínimo	Spread (bps)
Aaa/AAA	8,50	63
Aa2/AA	6,50	78
A1/A+	5,50	98
A2/A	4,25	108
A3/A-	3,00	122
Baa2/BBB	2,50	156
Ba1/BB+	2,25	200
Ba2/BB	2,00	240
B1/B+	1,75	350
B2/B	1,50	375
B3/B-	1,25	500
Caa/CCC	0,80	600
Ca2/CC	0,65	700
C2/C	0,20	800
D2/D	< 0,20	1200
Rating	ICR mínimo	Spread (bps)
Aaa/AAA	12,50	63
Aa2/AA	9,50	78
A1/A+	7,50	98
A2/A	6,00	108
A3/A-	4,50	122
Baa2/BBB	4,00	156
Ba1/BB+	3,50	200
Ba2/BB	3,00	240
B1/B+	2,50	350
B2/B	2,00	375
B3/B-	1,50	500
Caa/CCC	1,25	600
Ca2/CC	0,80	700
C2/C	0,50	800
D2/D	< 0,50	1200

Apêndice B

Referências e Leitura Adicional

Referências Bibliográficas

- [1] Damodaran, A. (2012). *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset*. 3ª ed. Wiley Finance.
- [2] Damodaran, A. (2009). *The Dark Side of Valuation*. 2ª ed. FT Press. [3ª ed., 2018]
- [3] Miles, J. A.; Ezzell, J. R. (1980). The Weighted Average Cost of Capital, Perfect Capital Markets, and Project Life: A Clarification. *Journal of Financial and Quantitative Analysis*, 15(3), 719–730.
- [4] Modigliani, F.; Miller, M. H. (1963). Corporate Income Taxes and the Cost of Capital: A Correction. *American Economic Review*, 53(3), 433–443.
- [5] Merton, R. C. (1974). On the Pricing of Corporate Debt: The Risk Structure of Interest Rates. *Journal of Finance*, 29(2), 449–470.
- [6] Fuller, R. J.; Hsia, C. C. (1984). A Simplified Common Stock Valuation Model. *Financial Analysts Journal*, 40(5), 49–56.
- [7] Hamada, R. S. (1972). The Effect of the Firm’s Capital Structure on the Systematic Risk of Common Stocks. *Journal of Finance*, 27(2), 435–452.
- [8] Damodaran, A. *Damodaran Online* — dados e tabelas atualizados anualmente. <https://pages.stern.nyu.edu/~adamodar/>
- [9] *ratmoney* — Biblioteca C++23 para aritmética monetária racional exata. <https://github.com/sheep-farm/ratmoney>